Day 123: Principal Component Analysis From Scratch: Variance Explained

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 18, Day 5 | Complete study guide

Watch: StatQuest: Principal Component Analysis (PCA), Step-by-Step (StatQuest with Josh Starmer) | PCA and SVD - the math explained simply (TileStats)

Script: `day123_pca.py` · every number in this guide is copied by program from `day123_run_output.txt`, the real output of that script.

PCA finds the directions along which data varies most: the eigenvectors of its covariance matrix. Written from scratch, with power iteration as a second route to the same answer, it matches scikit-learn exactly. On digits, 21 of 64 components hold 90% of the variance, and a kNN on just 10 components scores 0.9778 against 0.9907 on all 64. Two cautions measured here: unscaled wine's first component is just one feature (0.9981 of the variance), and PCA can throw away exactly the direction that matters (0.5250 accuracy on the first component, 1.0000 on the second).

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

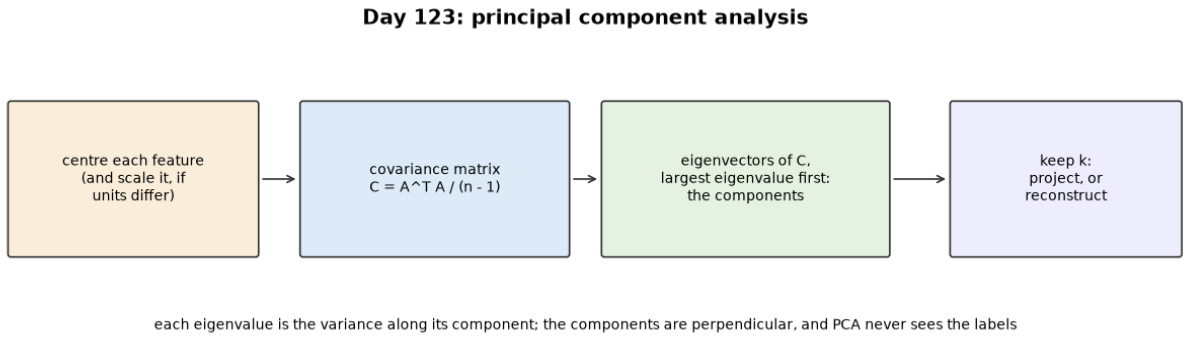
Principal component analysis (PCA) rotates the data so the first new axis, the first **principal component**, points along the direction of greatest variance; the second along the greatest variance perpendicular to it; and so on. The components are the **eigenvectors** of the covariance matrix, and each one's **eigenvalue** is the variance along it.

Keeping only the first k components gives a smaller table; the **variance explained** says how much of the spread survives.

REAL-WORLD SCENARIO

Photographing a pencil: from the side, its length is clear; end-on, it is a dot. PCA picks the viewpoints that show the most spread, so a 64-pixel image can be described by a handful of 'views'.

SYSTEM DIAGRAM



Centre, compute the covariance, take its eigenvectors in order of eigenvalue, keep k.

Today's part for the week's build

`day123_exercise.py` is the fit, the projection, the reconstruction and power iteration, checked by hand. Its real output:

ACTUAL OUTPUT

```
points on y = 2x: eigenvalues [12.5, 0.0]; first component [0.4472, 0.8944], by hand (1,
    2)/sqrt(5) = [0.4472, 0.8944]
axis-aligned cloud with spreads 3, 1, 0.5: eigenvalues [8.515, 0.991, 0.244] (by hand about 9, 1,
    0.25); components are the axes: True
all 3 components rebuild the data exactly: True; 1 component leaves a mean squared distance per
    point of 1.234 (by hand about 1 + 0.25)
power iteration: eigenvalues [8.515, 0.991, 0.244], the same as eigh: True; steps [14, 21, 2]
```

Data: digits (raw pixels), breast cancer and wine (standardised), bundled with scikit-learn; labels are used only to judge. Day 115's kNN and Day 119's k-means are imported; scikit-learn's PCA is an outside check.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: five points

DEFINITION — start here, no prior knowledge assumed

Five points along a rising line. The covariance matrix is 2 x 2, so its eigenvalues follow from a formula.

REAL-WORLD SCENARIO

Finding the direction a crowd is walking from a photograph of where people stand.

MATHEMATICS

$$C = A^T A / (n - 1), A = X - mean; \quad C\,v = \lambda\,v$$

$$2 \times 2: \lambda = (a + c)/2 \pm \sqrt{(((a - c)/2)^2 + b^2)} \text{ for } C = [[a, b], [b, c]]$$

IMPLEMENTATION

```
def pca_fit(X):
    mu = X.mean(0); A = X - mu
    C = A.T @ A / (len(X) - 1)
    vals, vecs = np.linalg.eigh(C)
    order = np.argsort(vals)[::-1]
    return mu, np.maximum(vals[order], 0), vecs[:, order]
def transform(X, mu, V, k):
    return (X - mu) @ V[:, :k]
def reconstruct(Z, mu, V, k):
    return Z @ V[:, :k].T + mu
```

ACTUAL OUTPUT

```
covariance [[2.5000, 2.1250], [2.1250, 1.8750]]
by hand: eigenvalues = (a + c)/2 +- sqrt(((a - c)/2)^2 + b^2) = 4.3354 and 0.0396; eigh gives
    4.3354 and 0.0396
first component (0.7568, 0.6536): at 40.8 degrees, along the line the points follow; it explains
    0.9909 of the variance
coordinates along it: [-2.4941, -1.4104, 0.0, 1.0836, 2.8209]
Actual output from this lesson's run.
```

What it means

The formula and eigh agree (4.3354 and 0.0396). The first component points at 40.8 degrees, along the points, and explains 0.9909 of the variance: one number per point, its position along that line, keeps almost everything.

WHY CHOOSE THIS?

A small, exact computation behind every result.

WHY NOT THIS? (TRADEOFFS)

Eigenvalues by formula only work in two dimensions; beyond that, an algorithm.

2. Correctness, and power iteration

DEFINITION — start here, no prior knowledge assumed

Digits against scikit-learn; then the top five components again by power iteration, which only multiplies a vector by the matrix, renormalises, and repeats.

REAL-WORLD SCENARIO

Shaking a box of mixed sand and pebbles: the largest pieces rise to the top first.

MATHEMATICS

$v \leftarrow Cv / |Cv|$ *until* v *stops changing;* $\lambda = v^T C v$; *then deflate:* $C \leftarrow C - \lambda\, v\, v^T$

after t *steps the error shrinks like* $(\lambda_2/\lambda_1)^t$

IMPLEMENTATION

```
def power_iteration(C, k, iters=5000, tol=1e-12, seed=0):
    rng = np.random.RandomState(seed); C = C.copy(); vals, vecs, steps = [], [], []
    for _ in range(k):
        v = rng.normal(size=len(C)); v /= np.linalg.norm(v)
        for t in range(1, iters + 1):
            w = C @ v; w /= np.linalg.norm(w)
            done = np.linalg.norm(w - v) < tol
            v = w
            if done:
                break
        lam = float(v @ C @ v); vals.append(lam); vecs.append(v); steps.append(t)
        C -= lam * np.outer(v, v)
    return np.array(vals), np.array(vecs).T, steps
```

ACTUAL OUTPUT

```
digits: explained-variance shares, largest gap from scikit-learn 8.3e-17; first 10 components, smallest |cos|
    with scikit-learn's 1.0000000000 (signs may differ)
power iteration, top 5: largest eigenvalue gap 4.8e-16 (relative); smallest |cos| with eigh's 1.00000000
  steps to converge: [295, 180, 84, 73, 155]; ratios of each eigenvalue to the next: [0.915, 0.866, 0.713,
     0.688, 0.85]
power iteration converges at the speed of that ratio: the closer two eigenvalues, the more steps
Actual output from this lesson's run.
```

What it means

Shares match scikit-learn to 8.3e-17, and the components agree up to sign. Power iteration finds the same five eigenvalues (4.8e-16 relative gap). The steps it needed, [295, 180, 84, 73, 155], follow the eigenvalue ratios [0.915, 0.866, 0.713, 0.688, 0.85] exactly in order: the closer an eigenvalue is to the next, the longer the two take to separate.

WHY CHOOSE THIS?

Power iteration needs only matrix-vector products, so it scales to huge sparse matrices.

WHY NOT THIS? (TRADEOFFS)

Slow when eigenvalues are close; eigh is the tool for small dense matrices.

3. Variance explained on three real datasets

DEFINITION — start here, no prior knowledge assumed

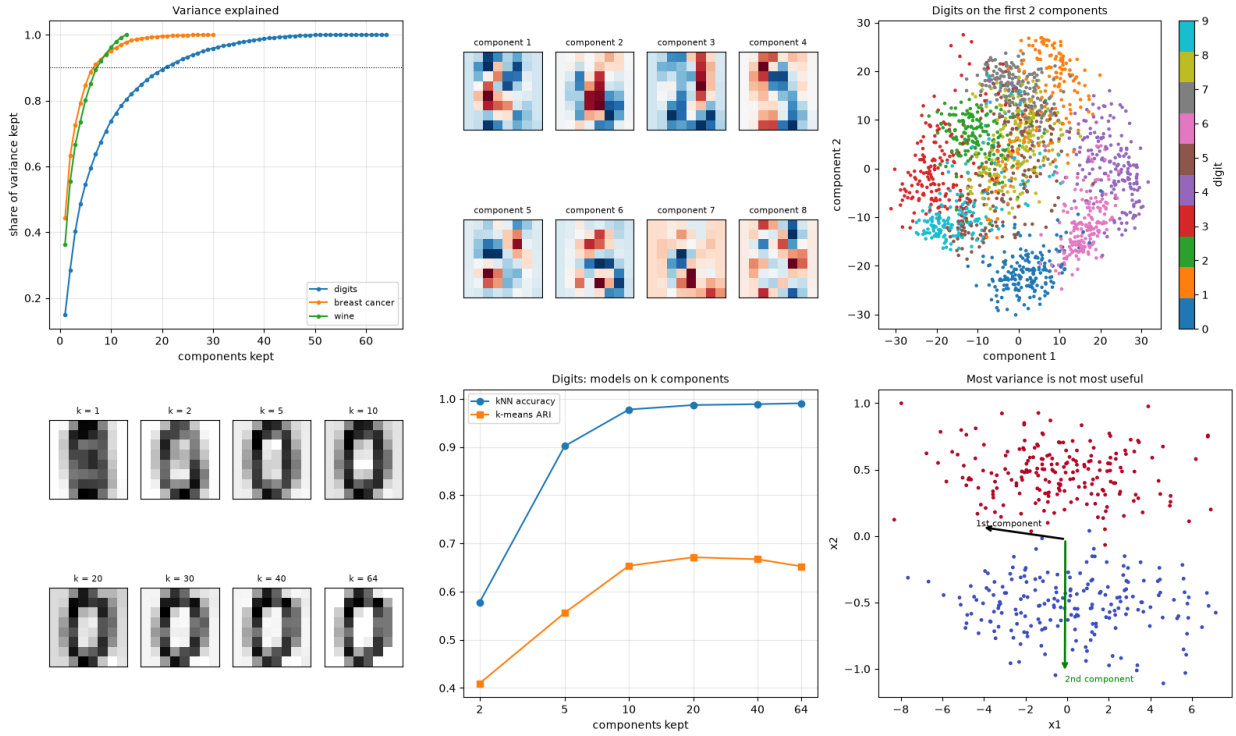
How many components are needed to keep 80%, 90%, 95% and 99% of the variance.

REAL-WORLD SCENARIO

How many camera angles it takes to show most of a sculpture.

ACTUAL OUTPUT

```
  data           features  1st share  for 80%  for 90%  for 95%  for 99%
  digits               64     0.1489       13       21       29       41
  breast cancer        30     0.4427        5        7       10       17
  wine                 13     0.3620        5        8       10       12
(breast cancer and wine standardised; digits as raw pixels)
Actual output from this lesson's run.
```



Top: cumulative variance explained; the first eight digit components as images (red positive, blue negative); digits on the first two components. Bottom: one digit rebuilt from k components; models on k components; the low-variance signal.

What it means

Digits are spread over many directions: the first component holds only 0.1489, and 90% needs 21 of 64. Breast cancer's 30 measurements are far more redundant: the first component holds 0.4427 and 7 components keep 90%. The faster the curve rises, the more the features repeat each other.

WHY CHOOSE THIS?

One table shows how compressible a dataset is.

WHY NOT THIS? (TRADEOFFS)

'Variance' is not 'information about the labels' (section 7).

4. Reconstruction

DEFINITION — start here, no prior knowledge assumed

Each digit is projected onto k components and rebuilt; the error is compared with the variance in the components dropped.

REAL-WORLD SCENARIO

Describing a face with a few features from a police sketch kit: more features, a closer likeness.

MATHEMATICS

$$\text{mean squared error} = (n-1)/n \times (\lambda_{k+1} + \ldots + \lambda_d) / d$$

ACTUAL OUTPUT

```
     k  variance kept  mean squared error per pixel  sum of the dropped eigenvalues / 64
     1         0.1489                       15.9777                              15.9777
     2         0.2851                       13.4210                              13.4210
     5         0.5450                        8.5424                               8.5424
    10         0.7382                        4.9143                               4.9143
    20         0.8943                        1.9843                               1.9843
    40         0.9882                        0.2215                               0.2215
    64         1.0000                        0.0000                               0.0000
the error left is exactly the variance in the directions dropped: PCA keeps the most variance any
    k directions can
Actual output from this lesson's run.
```

What it means

The two columns are equal to four decimals at every k: the error is exactly the variance in the dropped directions, so no other k directions could do better. With 20 components the error per pixel falls from 15.9777 (one component) to 1.9843, and the bottom-left images show a zero already recognisable at 5.

WHY CHOOSE THIS?

A guaranteed best linear compression.

WHY NOT THIS? (TRADEOFFS)

Only linear: curved structure needs more components than it really has dimensions.

5. Scaling

DEFINITION — start here, no prior knowledge assumed

Wine, whose 'proline' feature is measured in the hundreds while others are below 1, raw and standardised.

REAL-WORLD SCENARIO

Measuring a room in millimetres for the length and metres for the width, then asking which way it is longest.

ACTUAL OUTPUT

```
  raw          : 1st component explains 0.9981; its largest weights: 'proline' 1.000, 'magnesium'
      0.018; k-means on 2 components, ARI 0.3711
  standardised : 1st component explains 0.3620; its largest weights: 'flavanoids' 0.423,
      'total_phenols' 0.395; k-means on 2 components, ARI 0.8951
raw, the first component is almost exactly 'proline', the feature with the largest numbers;
    scaling decides what 'most variance' means
Actual output from this lesson's run.
```

What it means

Raw, the first component is proline alone (weight 1.000) and 'explains' 0.9981 of the variance, which only says proline has the largest numbers; k-means on two components scores 0.3711, as on Day 119. Standardised, the first component mixes several chemistry features, explains a more honest 0.3620, and k-means on two components scores 0.8951.

WHY CHOOSE THIS?

Standardising makes every feature count equally.

WHY NOT THIS? (TRADEOFFS)

It also inflates features that are mostly noise.

6. PCA before a model

DEFINITION — start here, no prior knowledge assumed

Digits: components learned from the training images only; kNN (k = 3) and k-means (k = 10) on 2 to 64 components.

REAL-WORLD SCENARIO

Planning a route on a map that keeps the main roads and drops the footpaths.

ACTUAL OUTPUT

```
   components  variance kept  kNN accuracy  k-means ARI
            2         0.2927        0.5778       0.4096
            5         0.5531        0.9019       0.5559
           10         0.7427        0.9778       0.6536
           20         0.8968        0.9870       0.6711
           40         0.9885        0.9889       0.6672
           64         1.0000        0.9907       0.6525
Actual output from this lesson's run.
```

What it means

Ten components (about three quarters of the variance) already give kNN 0.9778, against 0.9907 with all 64; two components give only 0.5778. k-means does slightly better on 20 components (0.6711) than on all 64 (0.6525): perhaps because dropping low-variance directions removes some noise from the distances (not tested further).

WHY CHOOSE THIS?

Smaller, faster models that lose little.

WHY NOT THIS? (TRADEOFFS)

The number of components is one more setting to choose.

7. Where PCA misleads

DEFINITION — start here, no prior knowledge assumed

Two classes made on purpose: separated along a narrow direction, while a wide direction is pure noise.

REAL-WORLD SCENARIO

Sorting apples from pears by size, when size varies a lot but says nothing; the colour, which varies little, is what matters.

ACTUAL OUTPUT

```
  the 1st component only  : variance kept 0.9687; kNN accuracy 0.5250
  both components         : variance kept 1.0000; kNN accuracy 0.9917
  the 2nd component only  : variance kept 0.0313; kNN accuracy 1.0000
PCA never looks at the labels: 'most variance' and 'most useful' are different questions
Actual output from this lesson's run.
```

What it means

The first component keeps 0.9687 of the variance and none of the signal: kNN on it scores 0.5250, chance. The second keeps 0.0313 and all of it (1.0000). PCA never sees labels; on real data this is rarely so extreme, but it is why the number of components should be chosen by the model's score, not by variance alone.

WHY CHOOSE THIS?

A clear warning, with known truth.

WHY NOT THIS? (TRADEOFFS)

Made on purpose; real data usually puts some signal in the large components.

Interview preparation — technical questions

Q: How do you compute PCA?

A: Centre, form the covariance, take its eigenvectors in order of eigenvalue (or use the SVD, Day 124).

Q: How many components do you keep?

A: Enough for the task: on digits 10 gave kNN 0.9778 of the 0.9907 possible; variance thresholds are a start.

Q: Must you standardise first?

A: When units differ, yes: raw wine's first component was proline alone (0.9981).

Q: What is power iteration?

A: Repeated multiplication by the matrix; it converges to the top eigenvector at the rate of the eigenvalue ratio.

Q: Does PCA help a classifier?

A: Sometimes, by removing noise; but it can also discard the signal (0.5250 against 1.0000 here).

Interview preparation — theoretical questions

Q: Why are the components the covariance's eigenvectors?

A: The variance along a unit vector v is $v^T Cv$, which is largest at the top eigenvector.

Q: What does an eigenvalue mean here?

A: The variance of the data along its component.

Q: Why is the reconstruction error the dropped eigenvalues?

A: The components are perpendicular, so the variance splits exactly between kept and dropped.

Q: Is PCA supervised?

A: No: it never sees labels, which is both its generality and its blind spot.

Key takeaway and what's next

KEY TAKEAWAY

PCA rotates data onto its directions of greatest variance and lets you keep the few that matter most, with an exact account of what is lost. On digits ten directions did almost all a classifier needed. But 'most variance' is defined by the units you measure in, and it is not the same as 'most useful': scale first, and choose the number of components by the result you care about.

- Matches scikit-learn (8.3e-17); power iteration agrees, with steps that follow the eigenvalue ratios.
- Digits: 21 components for 90% of variance; kNN 0.9778 on 10 components, 0.9907 on 64.
- Raw wine: 1st component = proline, 0.9981 of variance.
- Signal in a low-variance direction: 0.5250 on the 1st component, 1.0000 on the 2nd.

Suggested watch order, if starting from zero

- 1. StatQuest: Principal Component Analysis (PCA), Step-by-Step (StatQuest with Josh Starmer) — PCA built up step by step.
- 2. PCA and SVD - the math explained simply (TileStats) — the eigendecomposition and SVD routes side by side.

Day 124 preview

- **The singular value decomposition** and its relationship to PCA, with a real matrix-factorization demonstration.